The RHM coverage hypothesis, tested and refuted -- and a better window than U=1

Coverage does not explain the 1D/2D gap -- matching on it leaves 5.1x, worse than the 2.3x matching on n. But success peaks at k=11, below the root-reaching U1=15, and beats it by 18%.

THE OPEN QUESTION. P* = exp(Theta(n)) holds in both dimensions, the constant does not: x* = P*/|R_L| is 0.050 in 1D against 0.193 in 2D at matched n=16. Coverage was the standing explanation -- U=1 spans 94% of 1D, 56% of 2D.

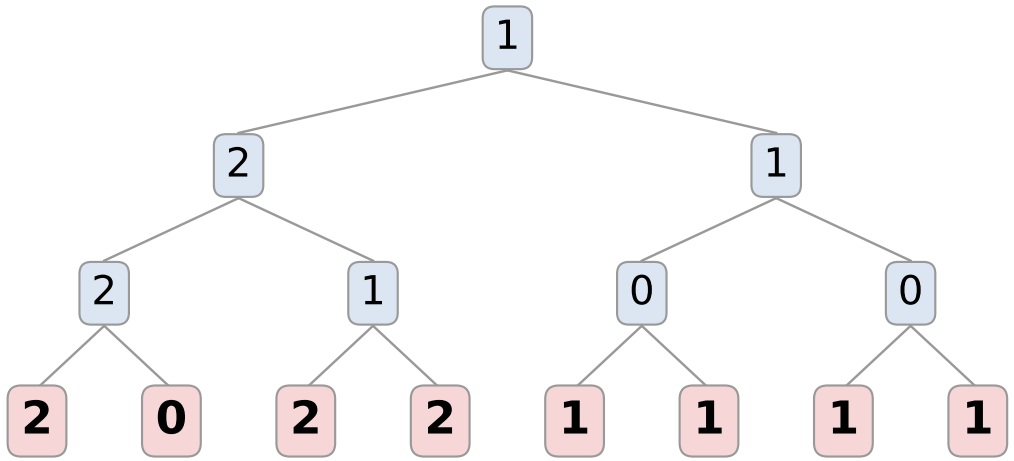
THE TEST. Sweep k at fixed n, dimension and grammar: the lever neither depth probe could pull, since there k and n moved together. 1D L=4 runs k = 3..17, and k=9 has coverage 9/16 -- exactly 2D L=2's. 2 432 runs, job 51058208.

REFUTED, SIGN BACKWARDS. x* rises with coverage (Spearman +1.00), and at matched coverage the dimensions are 5.1x apart -- worse than matched n. The table-size alternative fails too: 4.4x across dimensions, 61x across k.

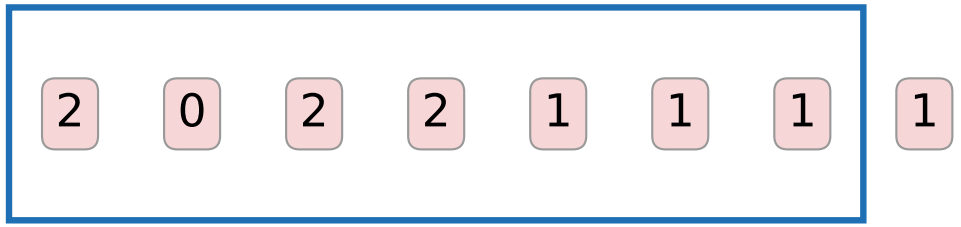
WHAT REPLACED IT. x* moves 25x across k, so "0.21" is a property of the U=1 convention, not of the RHM -- and U=1 is the wrong convention for generating.

1. Tokenization: the string, the tree, the window

v=3, s=2, L=3: root -> 8 leaf tokens



the sample the machine actually sees

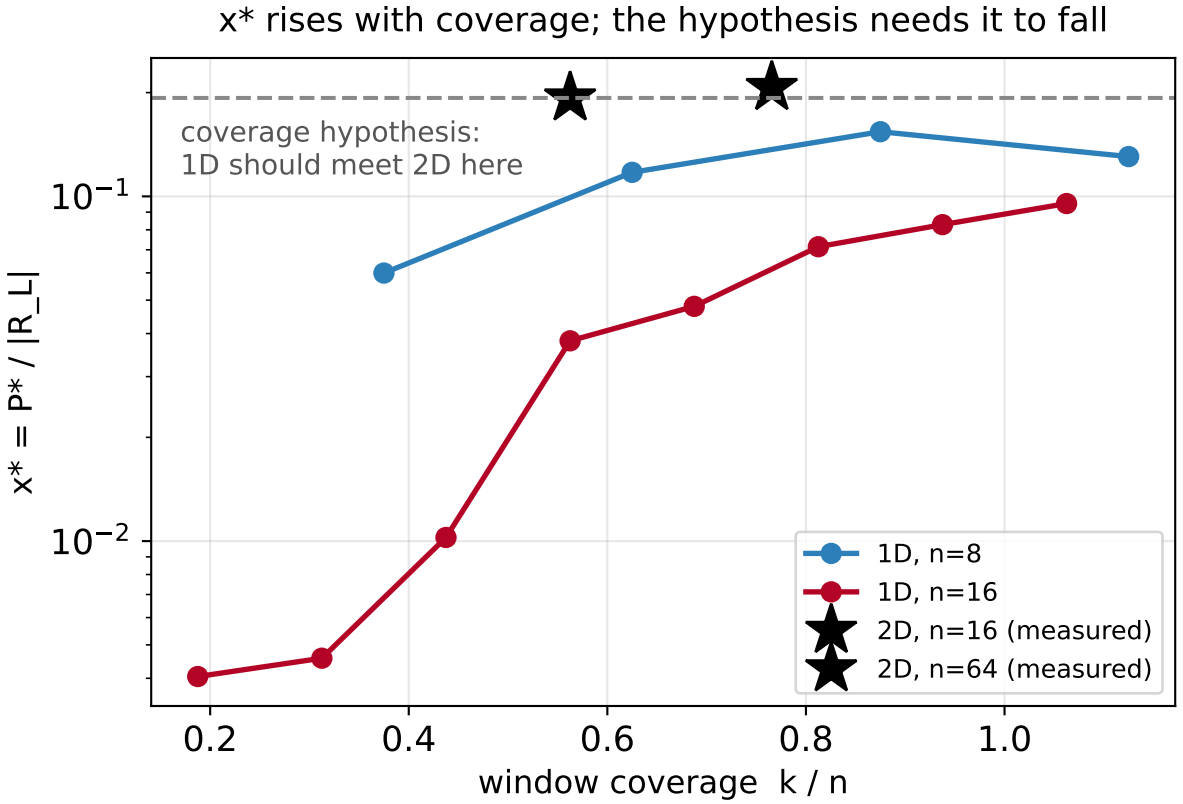


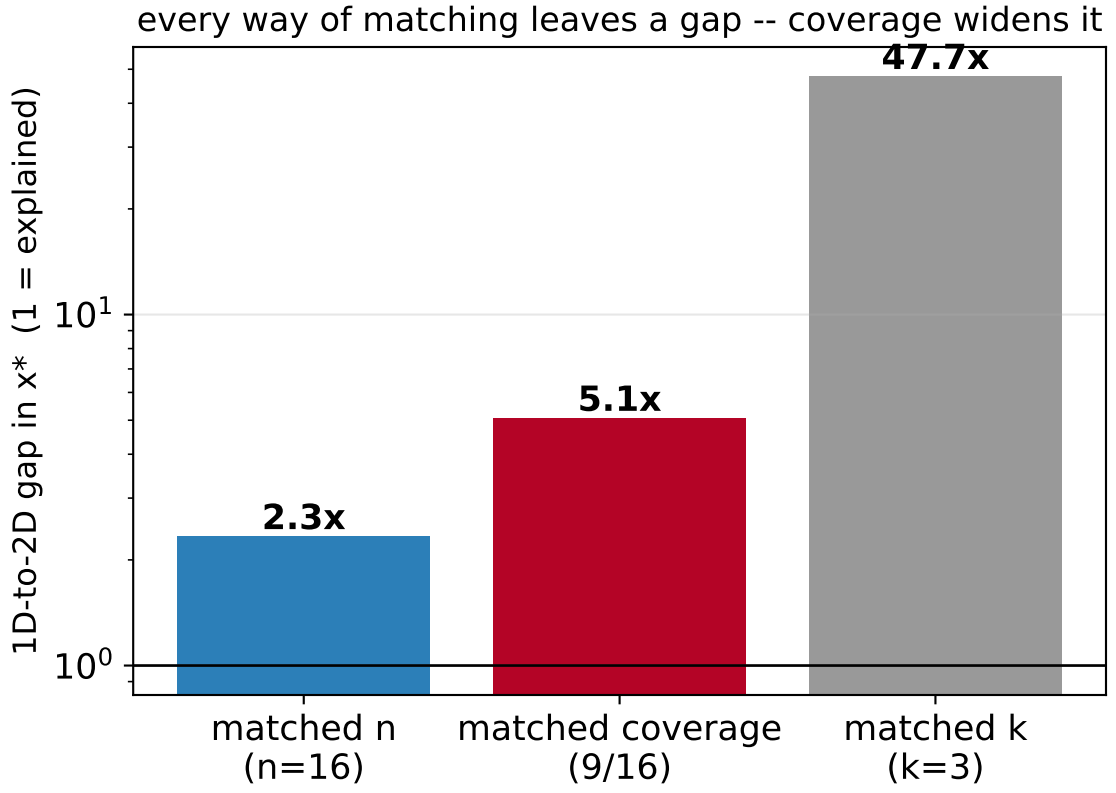
one U=1 window, k = 2s^(L-1) - 1 = 7

covers 7 of 8 sites = 88% of the sample

A token is one leaf symbol from the alphabet 0..v-1. One *sample* is the whole length-d string, d = s^L. The machine is a lookup table over length-k windows: for a masked site it returns the empirical distribution of the centre tokens of every training window that agrees on all observed sites. Off-grid sites get their own marker symbol v and count as observed, so a window at the border can only match training windows framed the same way.
k = 2s^(L-1) - 1 is the smallest window that fully contains the top-level block, i.e. the smallest one that can see the root. In 1D that is d - 1 at every depth, which is the whole story of this deck.

3. The test: sweep k at fixed n. Coverage is refuted, sign backwards

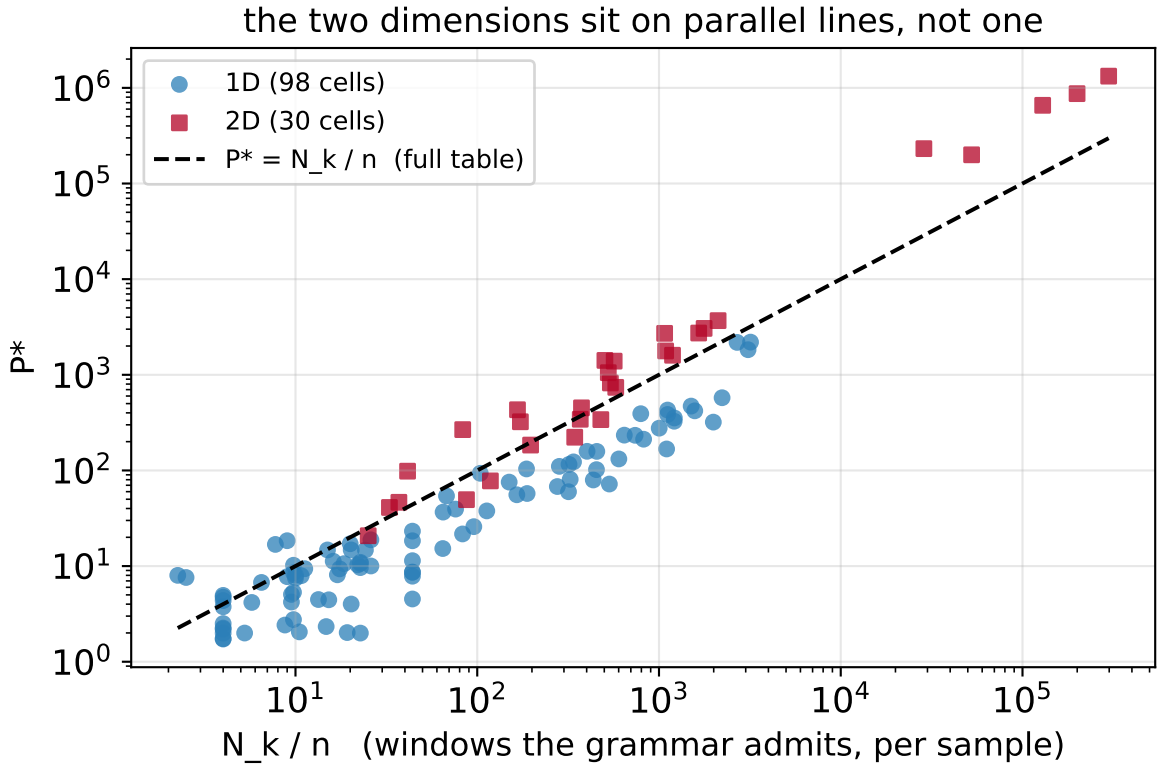


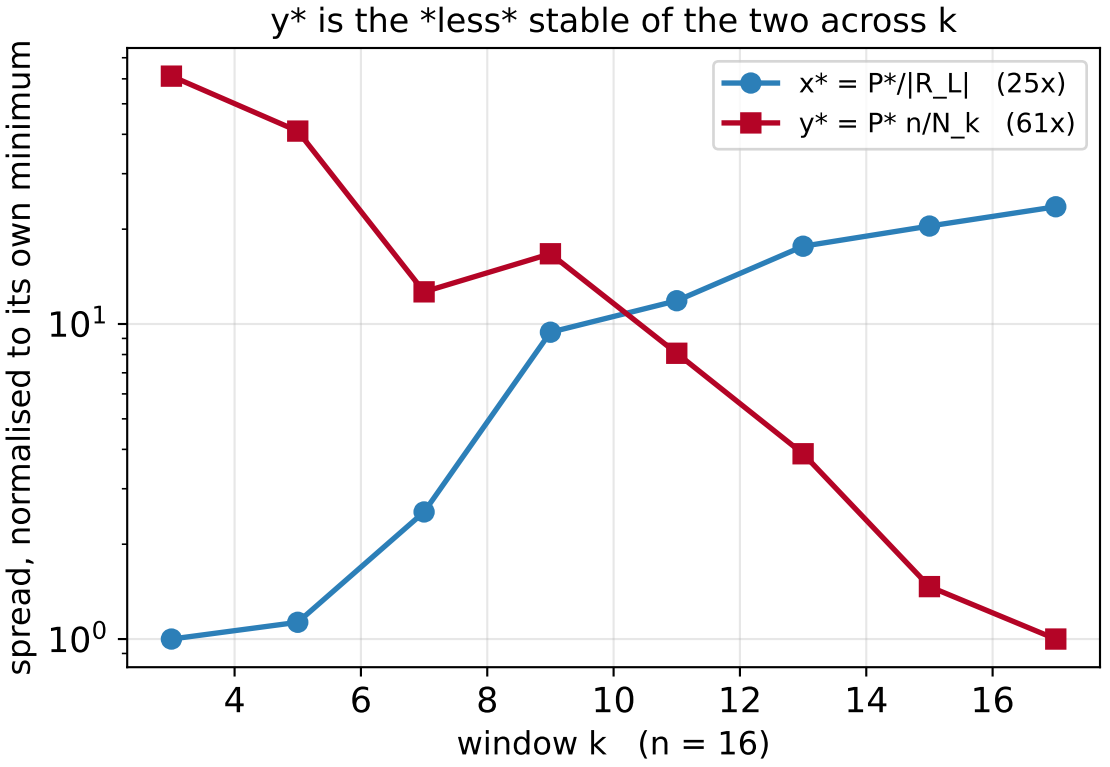


REGISTERED BEFORE THE RUN. Coverage drives x* -> x* rises as k falls, reaching ~0.19 at k=9, whose coverage 9/16 is exactly 2D L=2's. Seeing the root drives it -> x* flat for k >= U1, success collapsing below.

MEASURED. x* *falls* as k falls, monotonically: 0.004 at k=3 to 0.100 at k=17, a 25x spread, Spearman(coverage, x*) = +1.00 at n=16 and +0.80 at n=8. The hypothesis required the opposite sign. At matched coverage 1D gives 0.038 against 2D's 0.193 -- 5.1x apart, where matched n alone leaves 2.3x on these same cells (the depth probe, a wider cell mix, put it at 3.2x). Matching on coverage more than doubles the discrepancy it was introduced to explain.

4. The principled alternative fails too: it is not table size

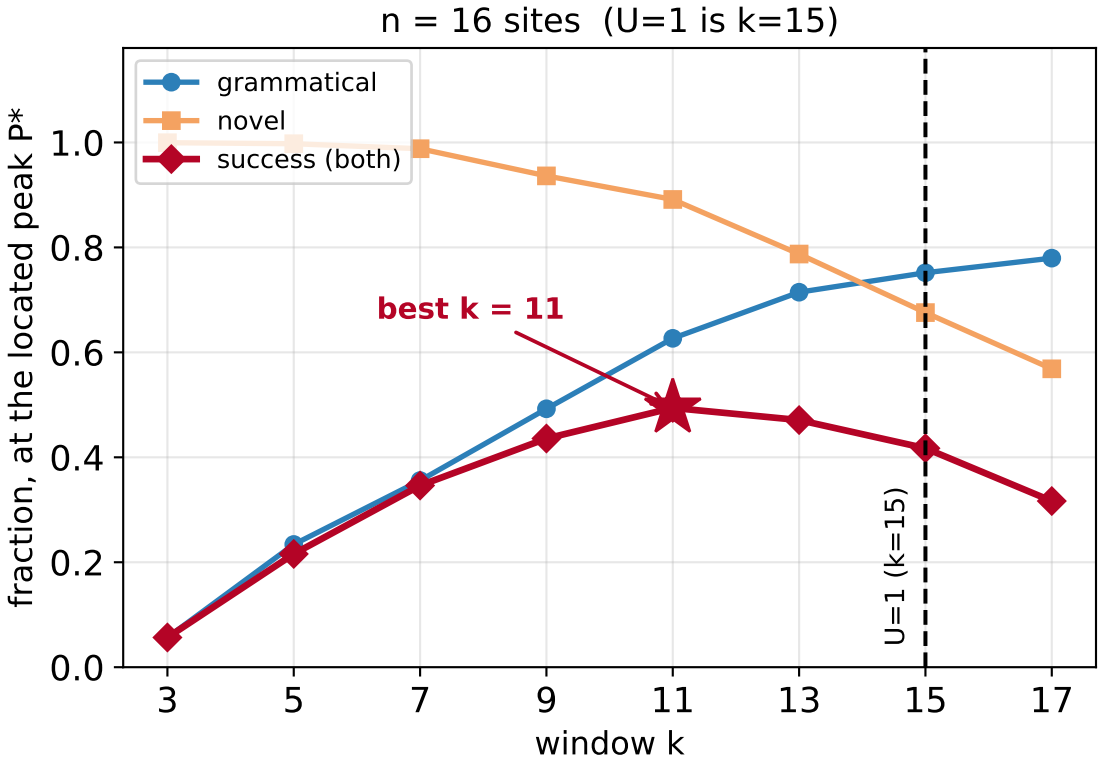


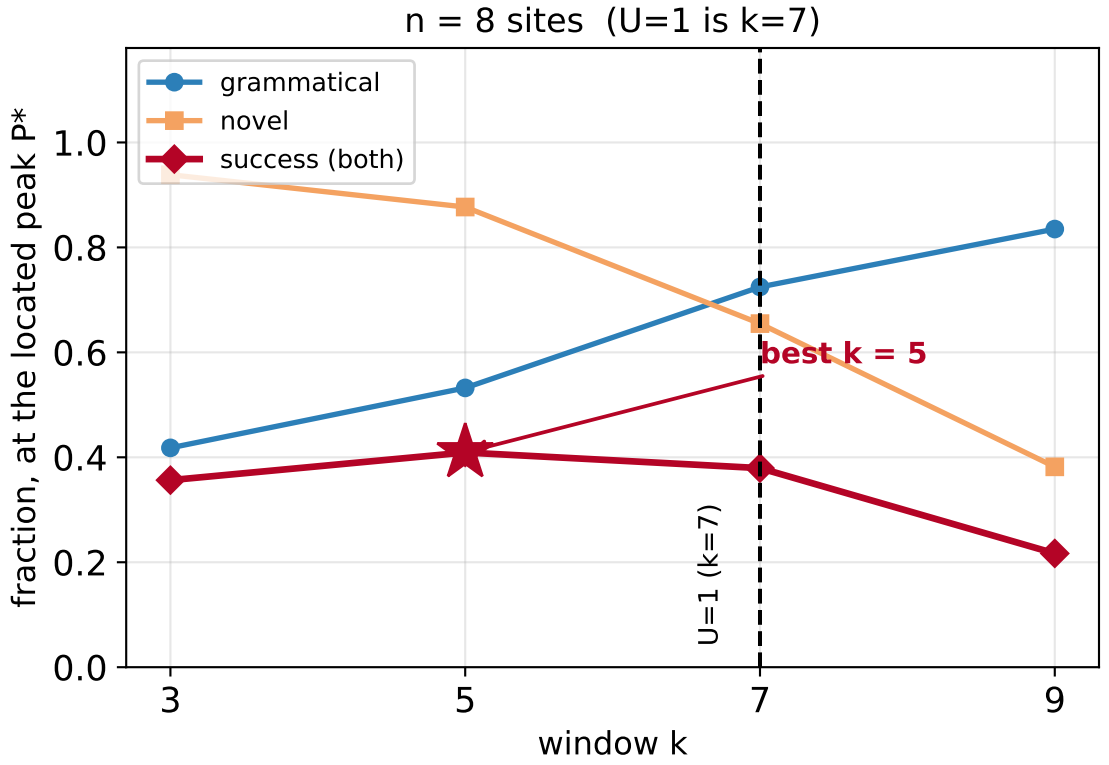


The machine never sees a sample; it sees n windows per sample, drawn from the N_k the grammar admits. So the natural law is P* ~ N_k/n, and N_k knows about window geometry -- the one thing that genuinely differs between 1D and 2D at equal n. N_k is exact here, enumerated over the whole grammar by odometer, not sampled.

It fails both ways. Across dimensions y* = P* n/N_k is 4.4x apart where x* is 3.2x (both on the depth-probe cells of this panel, not page 3's k-sweep cells, which put the x* gap at 2.3x); within 1D y* spreads 61x across k, x* 25x. Note y* crosses 1 between the dimensions: 2D peaks at y* = 1.5-4.5, *after* the training set has supplied one window per distinct window, while 1D peaks at 0.35 with the table a third full. Neither quantity is the collapse variable, and the 1D/2D offset survives as a genuine dimensional effect rather than a bookkeeping error.

5. The window optimum sits BELOW the root-reaching window





MECHANISM. Widening the window buys grammaticality and spends novelty. At n=16, across k = 3..17, grammaticality at the peak rises 0.06 -> 0.78 while novelty falls 1.00 -> 0.57. Success is their product, so it turns over -- at k=11, two rungs below the root-reaching U1=15, beating it by 18% (0.494 vs 0.417). At n=8 the best is k=5 against U1=7, +8%.

WHY IT MATTERS. U=1 is the window this whole program has used, chosen because it is the smallest that can see the root and therefore the smallest that can be fully grammatical. That reasoning optimises the wrong thing: a window that can always reach the root also reproduces its training strings, and hshunt's criterion scores a reproduced string as a failure. The useful window is the largest one that stays short of the root.

6. The same grammar at two windows, revealed one site at a time

v=2, s=2, L=4 (n=16 sites), m=2; |R_L| = 12544; P = 64 training strings. Alphabet {0,1}, "." = still masked.

window k = 11 (below U=1)

generation 1

```
................   site  2 -> 1   p(1)=0.17
..1.............   site  1 -> 1   p(1)=0.45
.11.............   site 15 -> 1   p(1)=0.53
.11............1   site 11 -> 0   p(1)=0.44
.11........0...1   site  0 -> 0   p(1)=0.00
011........0...1   site 13 -> 0   p(1)=0.58
...  final 0110011100000001
```

grammatical: yes **novel: yes**

generation 2

```
................   site 14 -> 1   p(1)=0.22
..............1.   site  4 -> 1   p(1)=0.39
....1.........1.   site  5 -> 0   p(1)=0.45
....10........1.   site  6 -> 0   p(1)=0.29
....100.......1.   site  0 -> 1   p(1)=1.00
1...100.......1.   site  8 -> 1   p(1)=0.50
...  final 1001100110111111
```

grammatical: NO **novel: yes**

window k = 15 (U=1, sees the root)

generation 1

```
................   site  2 -> 1   p(1)=0.17
..1.............   site  1 -> 1   p(1)=0.45
.11.............   site 15 -> 1   p(1)=0.53
.11............1   site 11 -> 0   p(1)=0.44
.11........0...1   site  0 -> 0   p(1)=0.00
011........0...1   site 13 -> 0   p(1)=0.58
...  final 0111001000000001
```

grammatical: yes **novel: NO**

generation 2

```
................   site 14 -> 1   p(1)=0.22
..............1.   site  4 -> 1   p(1)=0.39
....1.........1.   site  5 -> 0   p(1)=0.36
....10........1.   site  6 -> 0   p(1)=0.00
....100.......1.   site  0 -> 1   p(1)=1.00
1...100.......1.   site  8 -> 0   p(1)=0.25
...  final 1001100100000111
```

grammatical: yes **novel: NO**

Each row is the state before one reveal: the string so far, the site the sampler chose uniformly among those still masked, the token drawn, and the posterior the window put on "1". The first six of sixteen reveals are shown, then the finished string. Both columns use the same grammar, the same training set and the same random seed; only the window differs.
The k=15 column is the failure mode page 5 quantifies: with a window that spans all but one site, matching training windows pins almost the whole string, so the generation is grammatical and *not new*. The k=11 column leaves the sampler room to recombine.

7. What this changes, and what it does not

UNCHANGED, AND STILL THE DELIVERABLE. P* = exp(Theta(n)) with n the number of sites, measured as the peak of "novel AND fully grammatical" against training-set size. In 2D at the U=1 window, P* = 0.21 |R_L| (30 bracketed peaks, |R_L| spanning 49 152x, free exponent 0.965 +- 0.041). Nothing here touches that.

RETIRED. The coverage hypothesis, which was the last standing explanation for why the 0.21 is 2D-specific. It is refuted with the sign backwards: x* rises with coverage (Spearman +1.00), and matching on coverage puts 1D and 2D 5.1x apart where matching on n leaves 2.3x. The table-saturation alternative fails too, 4.4x across dimensions and a 61x spread across k. I have no mechanism for the 1D/2D offset and am no longer proposing one as if I did.

NEW, AND THE REASON THIS WAS WORTH RUNNING. x* is a steep function of the window: 25x across k = 3..17 at fixed n. So "0.21" was never a property of the RHM alone -- it is a property of the RHM *at the U=1 window*, and both dimensions had only ever been measured at that one convention. And U=1 is the wrong convention for generation: success peaks at k=11 against U1=15 (+18%) at n=16, and at k=5 against U1=7 (+8%) at n=8, because widening the window buys grammaticality with novelty.

HONEST LIMITS. Two depths, n=8 and n=16, and 1D only; the n=8 row leans on few cells after the density cut, since |R_L| >= 200 at v=2 means a grammar covering 78-100% of all 2^8 strings, where noise scores 1.00 on grammaticality. The +18% is a median over 11 rule seeds, not a single run, but it has not been checked in 2D, where the sub-U=1 windows that would test it produce <0.7% grammaticality and no locatable peak.

NEXT. The window optimum is the actionable finding and it points straight at the open question hshunt has already raised -- setting the patch size from the posterior-entropy floor rather than from the block structure. The AdaptiveWindow machine in the repo already chooses k per site and per step; it has never been scored against hshunt's novel-and-grammatical criterion, only against grammaticality. That is the next measurement.